Programmieren 1 (PR1)
(Einführung in die Programmierung: Java)

1. Eine erste Einführung in Java

WS 2026/27
Hans Grönniger

Bachelor-Studium
Angewandte Informatik
Mediendesigninformatik

Überblick

1. Eine erste Einführung in Java
   a. Programmiersprachen
   b. Algorithmus
   c. Ein erstes Java Programm
   d. Werkzeuge
   e. Struktur von Java-Programmen
   f. System.out.println und Strings
   g. Prozedurale Zerlegung mit statischen Methoden
   h. Bezeichner und Schlüsselwörter
   i. Kommentare
   j. Zeichnen komplexer Bilder mit statischen Methoden

Programm

Programm: Liste von Anweisungen für einen Computer

- Anweisungen sagen dem Computer wie er eine bestimmte Aufgabe erledigen soll.
- Vergleichbar mit einem Kochrezept: eine Ansammlung von Anweisungen, die dem Koch sagen, wie er die Speise zubereiten soll.
  - erforderliche Zutaten (die Daten)
  - Abfolge der Verfahrensschritte (der Prozess), der benötigt wird, um die Zutaten in einen Kuchen oder was anderes umzuwandeln.

Programmausführung

Programmausführung: Der Akt der Ausführung der Anweisungen

Programmiersprache

Programmiersprache: Systematische Menge von Regeln zur Beschreibung von Anweisungen in einem durch Menschen bearbeitbaren Format

- Natürliche Sprachen/Zahlen eignen sich nicht für eine genügend detaillierte und präzise Beschreibung von algorithmischen Computerberechnungen
  - zu vieldeutig
  - nicht formal genug
- In dieser Vorlesung lernen wir die Programmiersprache Java.



- Informationen über andere einflussreiche Programmiersprachen entnehmen Sie der ersten Leseaufgabe.

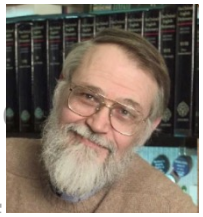
Moderne Programmiersprachen (Beispiele)

- *Prozedurale Sprachen*: Ein Programm ist eine Liste von Kommandos
  Pascal (1970, entwickelt für die Lehre),
  C (1972, für Betriebssysteme und Gerätetreiber)
- *Funktionale Programmierung*: Funktionen bilden Eingaben auf Ausgaben ab
  Lisp (1958) / **Scheme** (1975), **ML** (1973), **Haskell** (1990)
- *Objekt-orientierte Sprachen*: Programme nutzen interagierende „Objekte"
  Smalltalk (1980, Erste weitverbreitete objekt-orientierte Sprache),
  C++ (1985, „objekt-orientierte" Verbesserungen für C, Erfolgreich in der Industrie; Genutzt für die Programmierung von großen Betriebssystemen wie Windows),
  Java (1995, entwickelt für eingebettete Systeme und Webanwendungen; Läuft auf vielen Plattformen (Windows, Mac, Linux, Mobile Geräte ...)),
  C# (2001, u. a. Konzepte von C++ und Java, Gedacht für Windows-Anwendungen)
- *Multiparadigmen-Sprachen:* Mischung aus allen o. g. Konzepten in einer Sprache
  Fast alle o. g. Sprachen entwickeln sich zu Multiparadigmen-Sprachen.
  Javascript (1995, ursprünglich für dynamische Webseiten)
  Python (1991, Entwurfsziel: reduzierte Syntax)
  PHP (1995, für Webanwendungen)
  Ruby (1995, Skriptsprache; starke Verbreitung in Verbindung mit Webentwicklung mit Rails)

Moderne Programmiersprachen und ihre Erfinder



Niklaus Wirth



Brian Kernighan



Dennis Ritchie †



John McCarthy †



Adele Goldberg



Alan Kay



Bjarne Stroustrup



Anders Hejlsberg



James Gosling



Guido v. Rossum



Brendan Eich



Yukihiro Matsumoto



Rasmus Lerdorf

- *Prozedurale Sprache* – Programm ist … von Ko…
  Pascal (1970, entw… Lehre),
  C (1972, für Betrie… tra…
- *Funktionale Program…* – Funktionen bilden Eingaben … Ausgaben …
  Lisp (19…) / **Schem… ML** (1973), **Haskell** (1990)
- *Objekt-orientierte Sp…* – …ogramme nut… …agier… …bjekt…
  Smalltalk (1980, Ers… …jekt-o… Spra…
  C++ (1985, „objekt-o… …Verbesserunge… …utzt für die Programmierung von großen Betriebssys…
  Java (19…, entwickelt für ein… …end… …ft auf vielen Plattformen (Windows, Mac, L… …ik…
  C# (20…, u. a. Konzepte von … Java. Gedacht für Windo… …nge…
- *Multiparadigmen-Sprachen:* … aus… …g. Konzepte … Spra…
  Fast alle o. g. Sprac… …zu M… …gmen …
  Javascript (1995, ursprünglich für dynamis… …eiten)
  Python (19…, Entwurfsziel: … …te Synta…
  PHP (19…, für Webanwendu…
  Ruby (19… Skriptsprache; s… …erbreitung in Verbindung mit … …ung mit Rails)

Programmiersprachen-Generationen

- Was war vor den modernen Sprachen?
  - 1. Generation: **Maschinensprache**
    - **direkt auf einem Prozessor ausführbare binäre Zahlencodes**
  - 2. Generation: **Assembler**
    - **ersetzen die Zahlencodes der Maschinensprache durch symbolische Bezeichner**

      ```
      ADD  R21  R19
      ```
  - 3. Generation: **Höhere Programmiersprachen**
    - **führen Konzepte wie Variablen ein, um leichter verständlichen Quelltext schreiben zu können**

      ```
      int Summe; Summe = 3 + 4;
      ```
  - 4. Generation: **Viertgenerationssprachen**
    - **bieten einfache Sprachmittel zur Auslösung komplexer Operationen (bspw. Datenbankzugriffe)**

Vakatseite

Problem – Algorithmus - Programm

Mensch

Umgangssprache
- komplex
- mehrdeutig

menschl. Tätigkeit

Algorithmische Sprache
- semiformal
- noch nicht vollständig detailliert

menschl. Tätigkeit

Programmiersprache
- exakter Formalismus
- meist genormt
- unabhängig von einem bestimmten Prozessor

Compiler (Übersetzer)

Computer

Maschinensprache
- primitiv
- abhängig vom jeweiligen Prozessor

Problem

Algorithmus

Programm (Quelltext/Prog.-Sprache)

Maschinenprogramm

Algorithmus

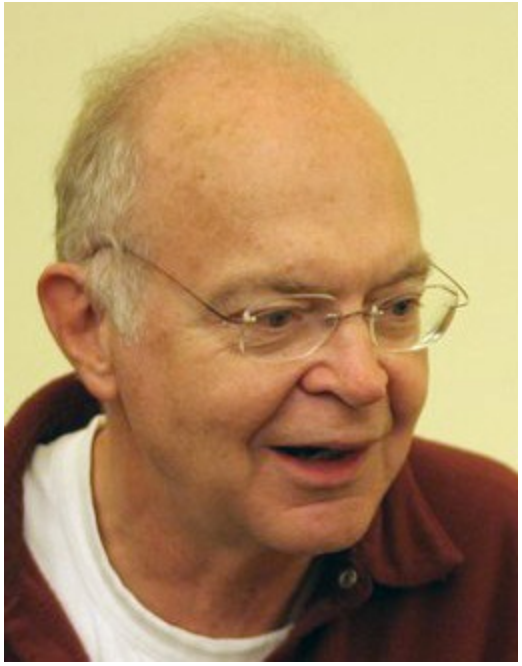
Algorithmus: Schritt-für-Schritt-Beschreibung zur Lösung eines Problems

- Ein Algorithmus hat Eingaben und Ausgaben
- Beispiel: „Kaffee kochen“
  1. Wasser, Kaffeefilter und Kaffeepulver bereitstellen
  2. Wasser in die Kaffeemaschine einfüllen
  3. Filter einlegen und Pulver einfüllen
  4. Maschine einschalten
  5. …

Algorithmus detailliert genug?

- Problem: „normale“ Menschen denken nicht detailliert genug
- Ist Schritt 2 detailliert genug?
  1. …
  2. Wasser in die Kaffeemaschine einfüllen
  3. …
- Ist das hier detailliert genug?
  2. Wasserhahn aufdrehen
  3. Einen 1-Liter-Behälter aus dem Schrank nehmen
  4. Den Behälter unter den Wasserhahn halten, bis er so voll ist, dass oben noch ca. 1 cm bis zum Rand ist
  5. Wasserhahn zudrehen
  6. Mit dem Behälter zur Kaffeemaschine gehen
  7. Inhalt des Behälters in die Kaffeemaschine gießen (in das Wasserfach!) so lange, bis das Wasserfach gefüllt ist
  8. Behälter im Waschbecken entleeren, abtrocknen und in den Schrank stellen

Wichtiger: Denken über Algorithmen



It has often been said that a person does not really understand something until he teaches it to someone else. Actually, a person does not really understand something until after teaching it to a computer, i.e., express it as an algorithm.

Donald Knuth

Formulierung von Algorithmen

- Wie schreibt man einen Algorithmus hin?
- Ziele:
  - In für Menschen verständlicher Form
  - So präzise wie möglich (Missverständnisse vermeiden)

Formulierung von Algorithmen

Pseudocode: Ein in strukturierter natürlicher Sprache geschriebener Algorithmus

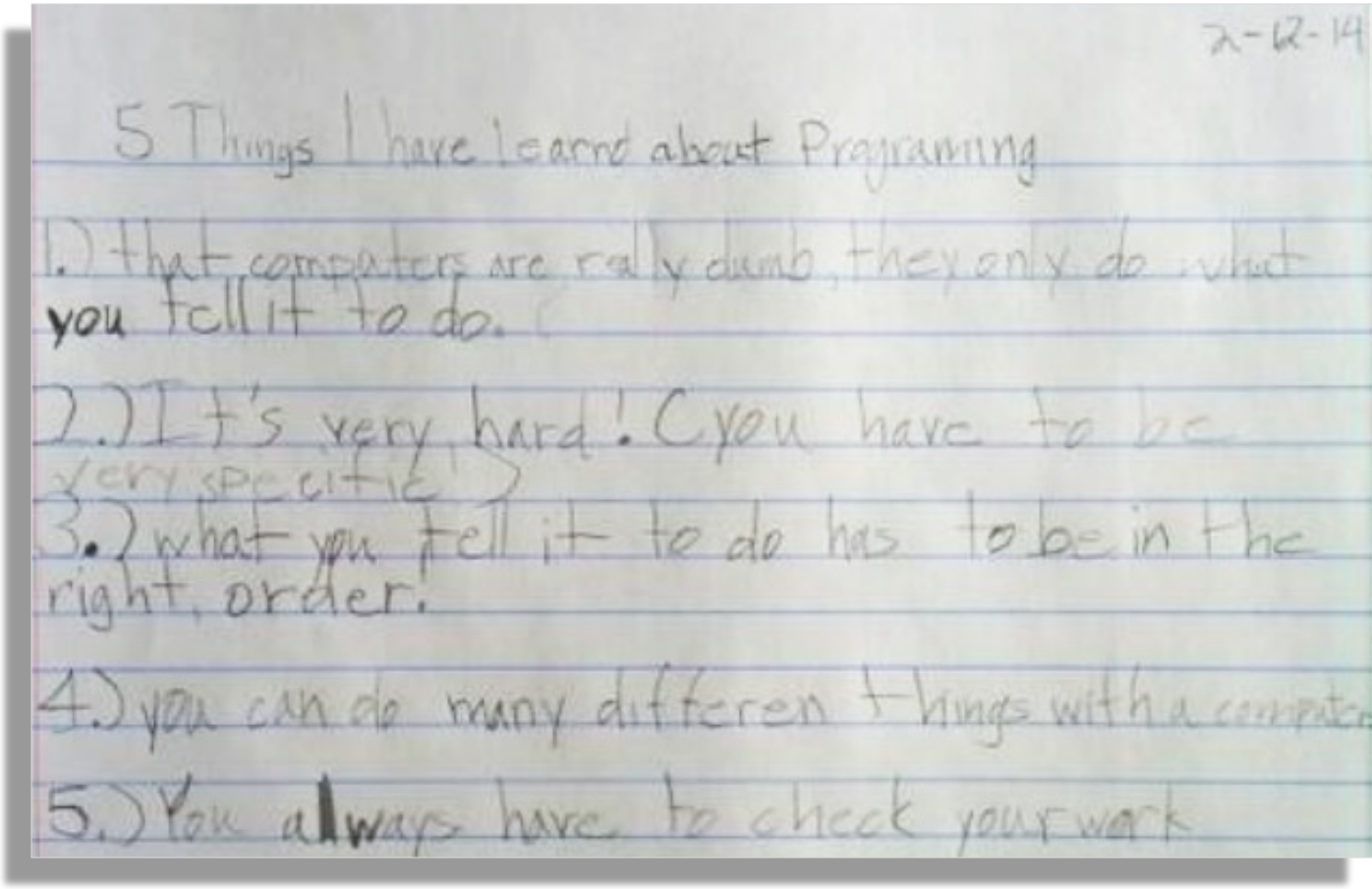
- Beispiel

Drucke 12 Sternchen.
Wiederhole für 5 Zeilen {
Drucke ein Sternchen.
Drucke 10 Leerzeichen.
Drucke ein Sternchen.
}
Drucke 12 Sternchen.

```
************
*          *
*          *
*          *
*          *
*          *
************
```

The essence of programming ... ☺

Eine Siebenjährige notiert nach einer Stunde mit Hopscotch, einer visuellen Programmiersprache:



Quelle: https://twitter.com/fjsteele/status/434249935316615168

Andere Formulierungen von Algorithmen



- Andere Möglichkeiten zur Formulierung von Algorithmen entnehmen Sie den Leseaufgaben.

Vakatseite

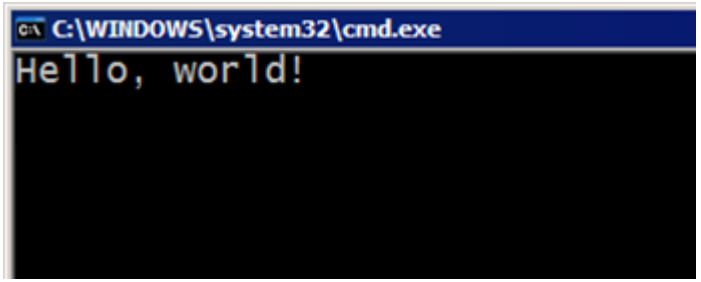
Ein erstes Java-Programm „*Hello World*"

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

Code oder Quellcode: Die Abfolge der Anweisungen eines Programms.

- Der Code dieses Programms weist den Computer an, die Nachricht "Hello, world!" auf dem Bildschirm anzuzeigen.

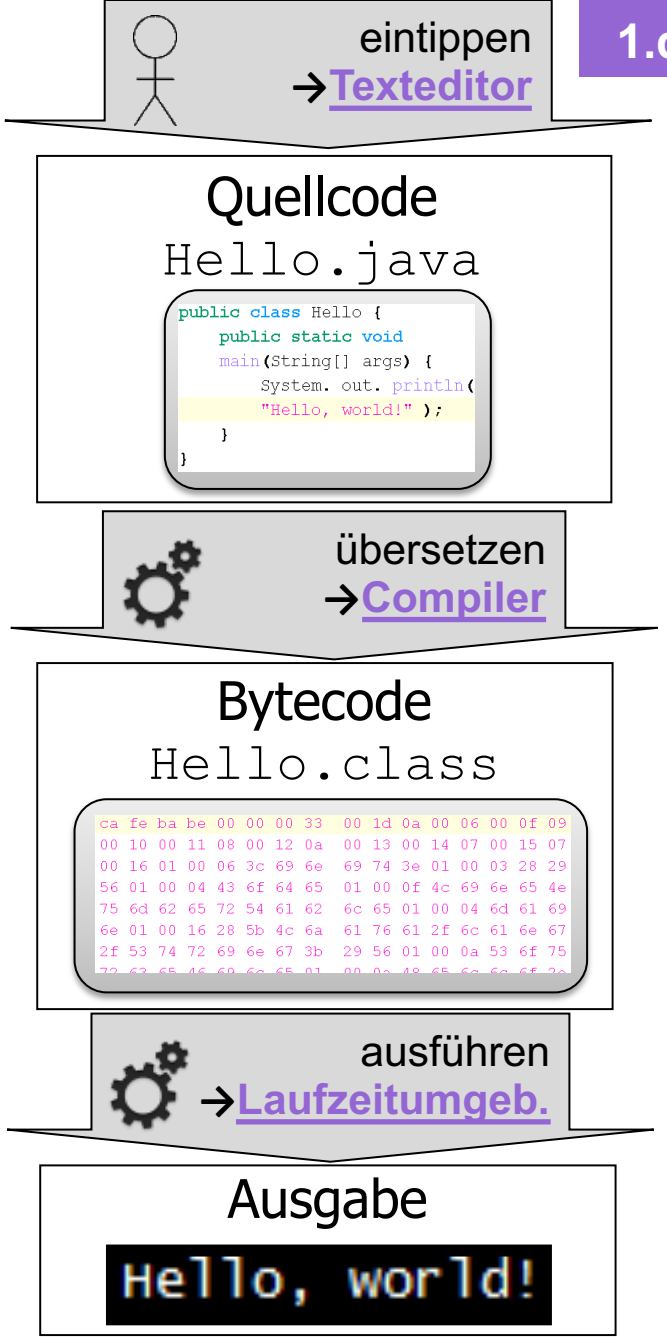
Console: Textfenster, in dem die Ausgabe angezeigt wird.



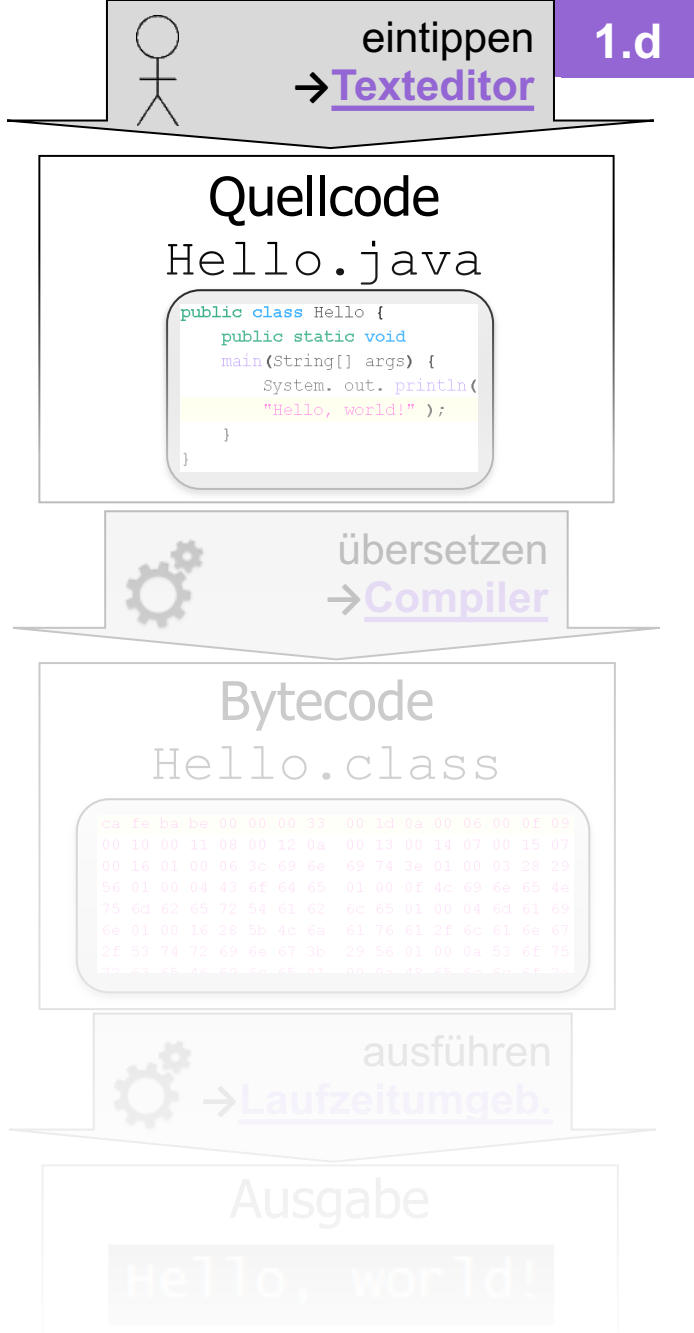
Vakatseite

Werkzeuge

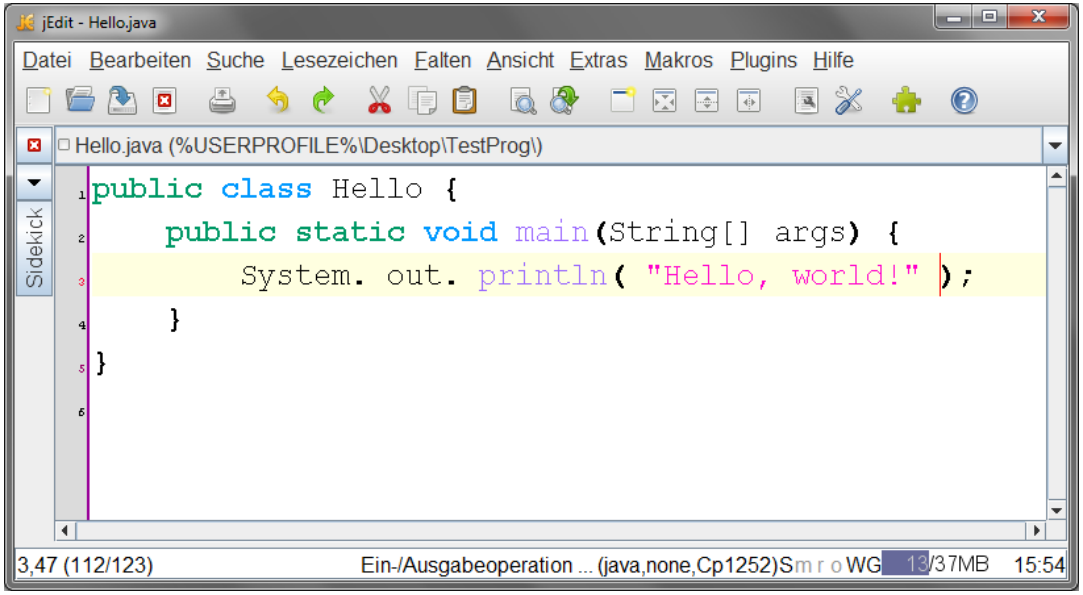
- Texteditor: Programm eintippen
- Compiler: Programm übersetzen
- Laufzeitumgebung: Programm ausführen



Texteditor



- Verschiedene Produkte verfügbar.
- **jEdit**:



- emacs, notepad, wordpad, textpad, vi, kate, kedit, …
- Weitere: http://de.wikipedia.org/wiki/Liste_von_Texteditoren

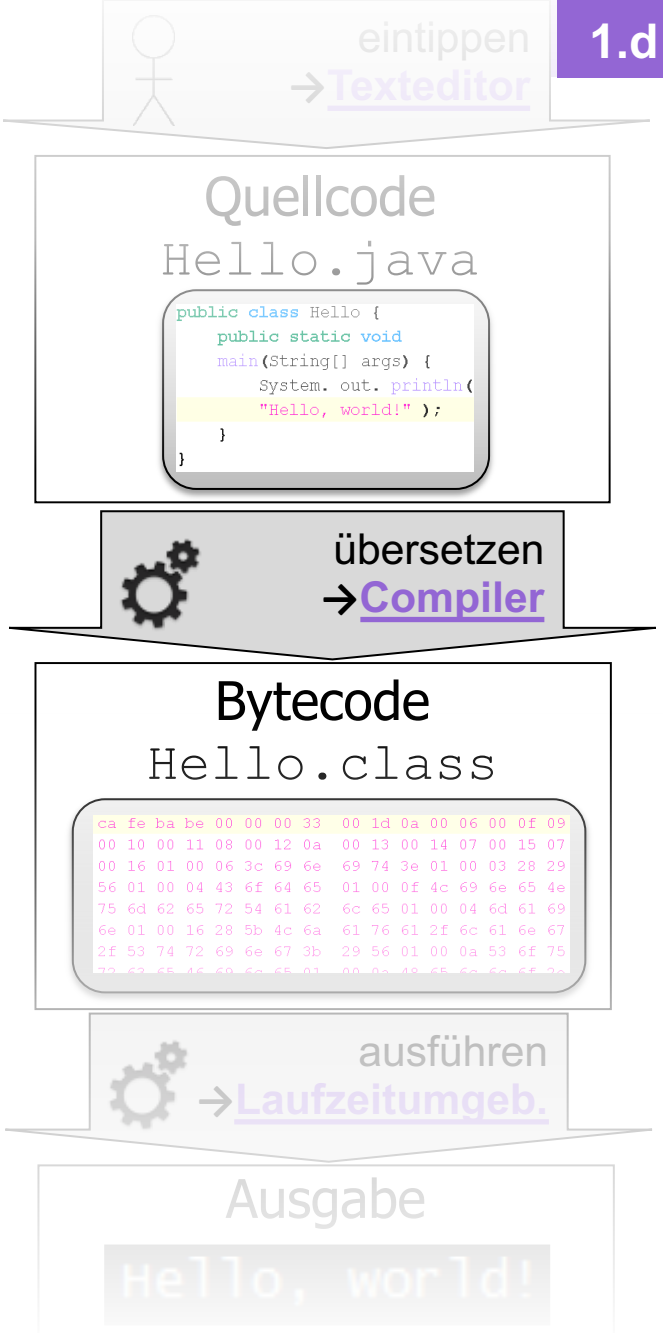
Texteditor

- Empfehlung für die nächsten Wochen: jEdit
  - Überschaubarer Umfang
  - Auf vielen Plattformen verfügbar.
    Zu Hause das gleiche Werkzeug wie im Rechnerpool
  - Syntax-Hervorhebung
  - Automatische Code-Vervollständigung
  - Kostet: nichts

Compiler

Compiler / Übersetzer: Übersetzt ein in einer Programmiersprache geschriebenes Programm in eine andere Sprache

- Das JDK (Java development kit) enthält einen Java Compiler
- Der Java Compiler konvertiert den Quellcode in ein Format namens **Bytecode**. Bytecode kann auf vielen verschiedenen Computern ausgeführt werden.
- Dabei: Überprüfung des Programms auf syntaktische Richtigkeit
- Starten durch **`javac Hello.java`**

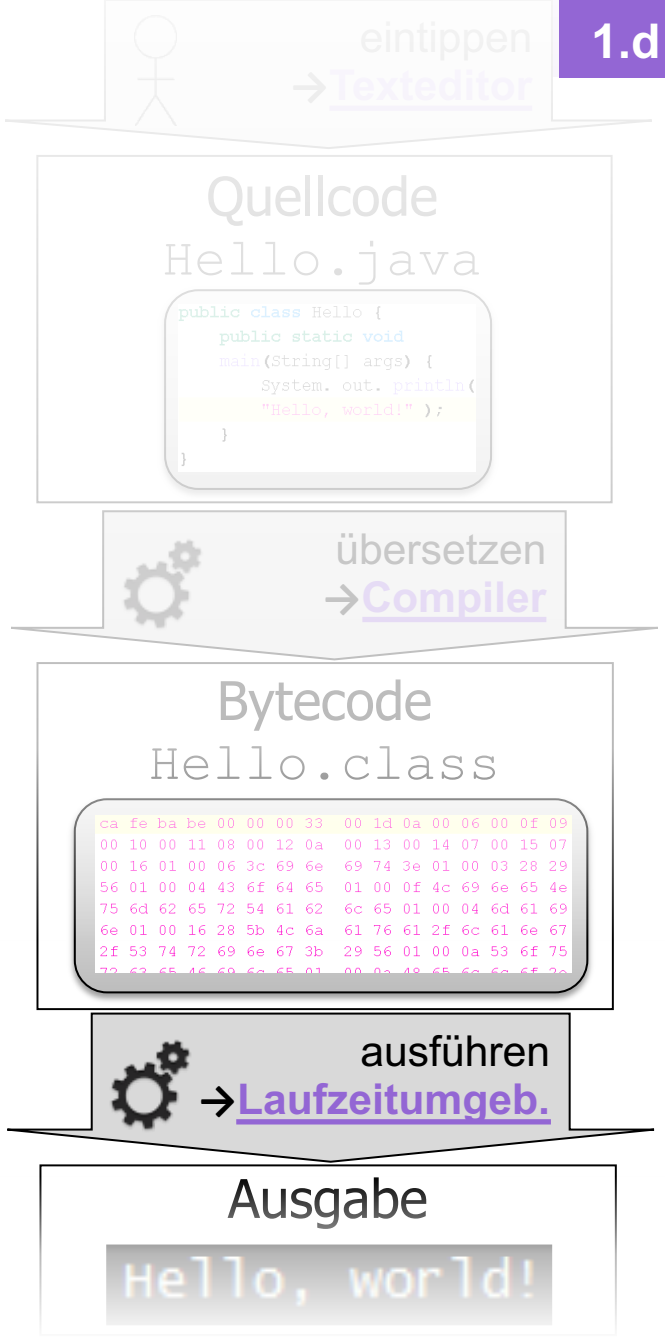


Laufzeitumgebung

JVM / Java virtual machine: Ein theoretischer Computer, dessen Maschinensprache der Java Bytecode ist.

- JVM dient als Schnittstelle zum eigentlichen Prozessor/Betriebssystem
- JVM gibt es auf unzähligen Plattformen
- JVM ist i. d. R. selbst in C und C++ programmiert.
- Eine Implementierung der JVM ist in der „Laufzeitumgebung" (Java Runtime Environment, JRE) enthalten.
- Starten durch **`java Hello`**

Java runtime / JRE: Ein Programm, das übersetzte Java class-Dateien (Bytecode) ausführt.



Mächtige Werkzeuge

IDE – Integrated development environment:
Software, die den Editier-Compile-Zyklus unterstützt.

- kontextsensitiver Editor
- „Designer" zum Gestalten graphischer Benutzungsoberflächen
- Hilfesystem für die Programmiersprache
- Verwaltung großer Projekte

- verschiedene Produkte für Java:
  - Eclipse **-> PR2**
  - NetBeans
  - IntelliJ IDEA
  - Weitere: http://de.wikipedia.org/wiki/Liste_von_Integrierten_Entwicklungsumgebungen

Vakatseite

Ein anderes Java Programm

```
public class Hello2 {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
        System.out.println();
        System.out.println("This program produces");
        System.out.println("four lines of output");
    }
}
```

- Dieser Programmcode gibt die Anweisung, vier Nachrichten auszugeben
- Die Ausgabe ist:

```
Hello, world!

This program produces
four lines of output
```

Struktur von Java Programmen

```
public class <name> {
    public static void main(String[] args) {
        <statement>;
        <statement>;
        ...
        <statement>;
    }
}
```

- Jedes Java-Programm besteht aus einer **Klasse**
  - die eine **Methode** namens `main` enthält
    - die wiederum **Statements** (Anweisungen) enthält, die ausgeführt werden sollen.

Struktur von Java-Programmen

Klasse: Ein grundlegendes Programmelement, aus dem jedes Java-Programm besteht.

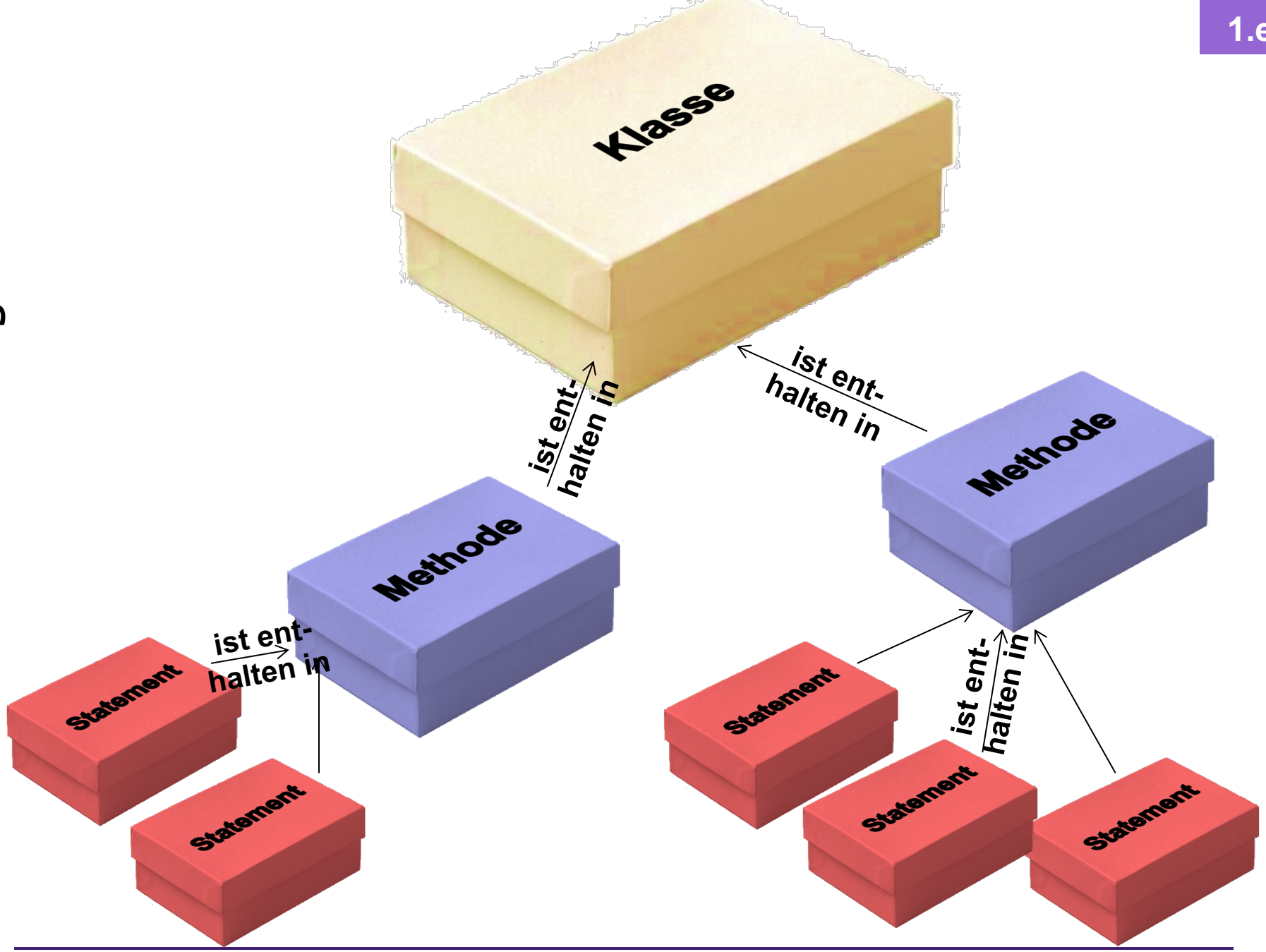
- Ein Programm kann aus einer oder mehreren Klassen bestehen.

Statement: Eine ausführbare Anweisung für den Computer.

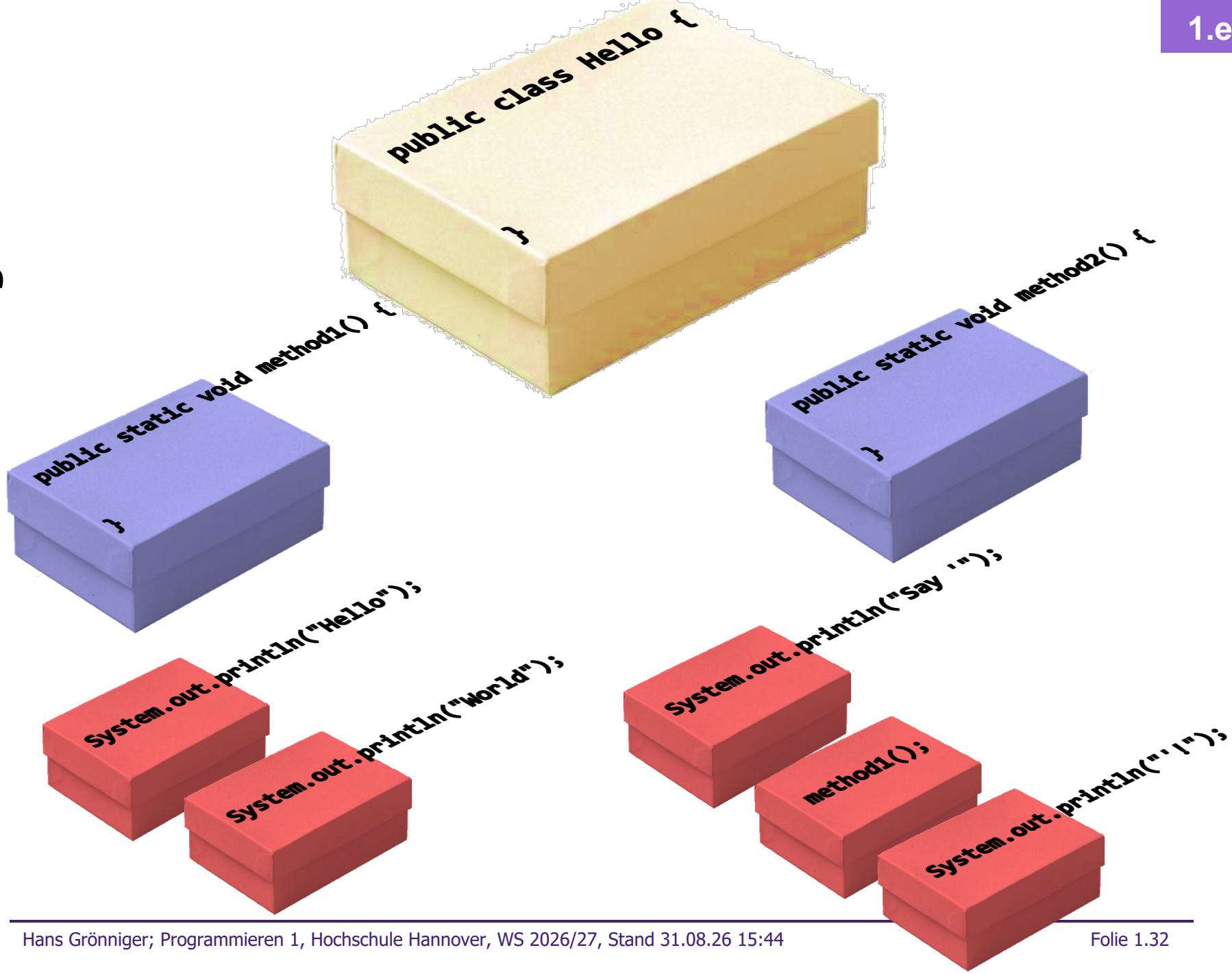
Methode: Eine benannte Folge von Statements, die als Ganzes ausgeführt werden.

- Die besondere Methode `main` enthält den Code, der ausgeführt werden soll, wenn das Programm startet
- Eine Klasse kann mehrere Methoden enthalten.

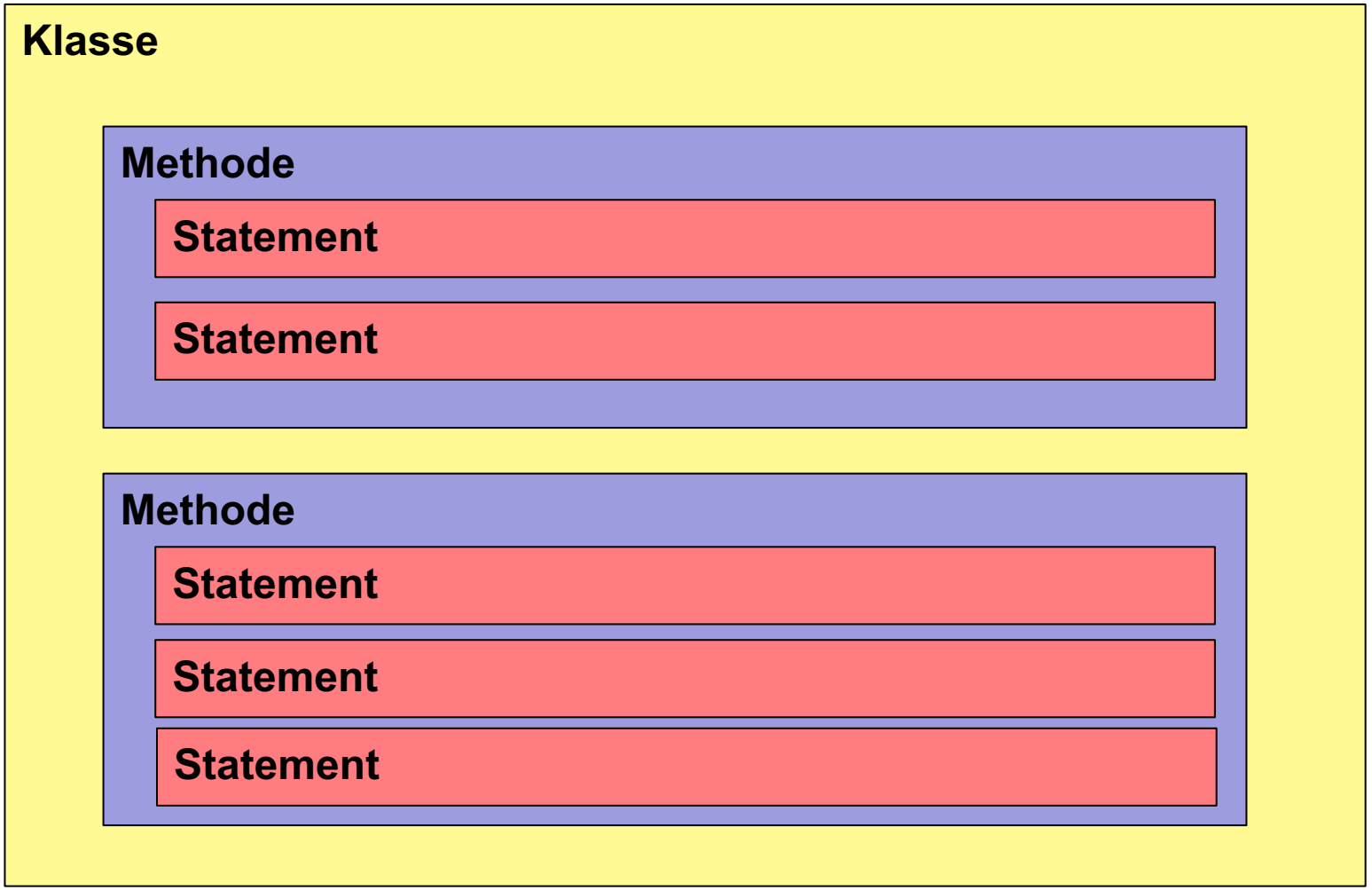
Struktur von Java-Programmen



Struktur von Java-Programmen



Struktur von Java-Programmen



Struktur von Java-Programmen

```
public class Hello {

    public static void method1() {
        System.out.println("Hello");
        System.out.println("World");
    }

    public static void method2() {
        System.out.println("Say '");
        method1();
        System.out.println("'!");
    }
}
```

Programm-Syntax

Syntax: Regeln für die Bildung von wohlgeformten Anweisungen in einem Programm.

- Von der inhaltlichen Bedeutung wird hierbei abgesehen
- Einige Regeln der Java-Syntax:
  - Jedes Statement endet mit einem Semikolon ;
  - Der Inhalt einer Klasse oder einer Methode steht zwischen zwei geschweiften Klammern { }

Syntaxfehler

Syntaxfehler / Compilerfehler: Ein Verstoß gegen die Regeln in einem Programm, der die Übersetzung fehlschlagen lässt.

```
1  public class Hello {
2      pOOblic static void main(String[] args) {
3          System.owt.println("Hello, world!")_
4      }
5  }
```

Ausgabe des Compilers:

```
HelloError.java:2: error: <identifier> expected
     pooblic static void main(String[] args) {
             ^
HelloError.java:3: error: ';' expected
         System.owt.println("Hello, world!")
                                            ^
2 errors
```

Syntaxfehler beheben

- Fehlermeldungen helfen nicht immer beim Verständnis des Fehlers

```
HelloError.java:2: error: <identifier> expected
     pooblic static void main(String[] args) {
               ^
```

- Netter wäre: „Tippfehler in `public`“

- Die Zeilennummer hinter dem Doppelpunkt hilft bei der Identifizierung.
- Ggf. steckt der Fehler auch in der Zeile davor oder danach.

Details zur Struktur von Java-Programmen

- Weitere Details („typische Programmierfehler“ und „Programmierkonventionen“) entnehmen Sie den Leseaufgaben.



Vakatseite

System.out.println

- `System.out.println` : Ein Statement, das den Computer anweist, eine Zeile auf der Console auszugeben.

- Zwei Varianten:

  `System.out.println("`***<Message>***`");`

  Gibt die gegebene Nachricht als eine Textzeile auf der Console aus.

  `System.out.println();`

  Gibt eine leere Zeile auf der Console aus.

Strings - Zeichenketten

String / Zeichenkette: Eine Folge von Zeichen, die ausgegeben oder vom Programm verarbeitet werden.

- Manchmal auch "String Literal" genannt.
- Strings in Java beginnen und enden mit Anführungszeichen (")

- Beispiele:

```
"hello"
"This is a string"
"This, too, is a string.   It can be very long!"
```

Details über Strings

- Ein String darf nicht über mehrere Zeilen gehen.[1]

```
"Dies ist ein
ungültiger String."
```

- Ein String darf kein Anführungszeichen `"` enthalten
  (Ein einfaches Anführungszeichen `'` ist erlaubt)

```
"Dies ist ein "ungültiger" String."
"Dieser hier ist 'okay'."
```



- Weitere Details entnehmen Sie den Leseaufgaben.

[1] Ab Java 15 gibt es die Möglichkeit von mehrzeiligen Strings (sog. *Text Blocks*). Details: s. Leseaufgabe.

Vakatseite

Algorithmus „Kekse backen“

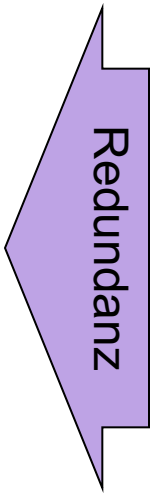
- Wie backt man Kekse?
- Wie lautet der „Kekse backen“-Algorithmus?

(Unstrukturierter) Algorithmus „Kekse backen“

- Margarine und Zucker schaumig rühren.
- 1 Eigelb unterrühren.
- Mehl dazugeben und zu einem glatten Teig verkneten.
- 1 Stunde kalt stellen.
- Teig ausrollen und Kekse ausstechen.
- Kekse auf Backblech legen und 10 Minuten backen.
- Zutaten für den Zuckerguss mischen
- Kekse mit Zuckerguss bestreichen

Wie backen wir zwei Bleche Kekse nacheinander?

- Margarine und Zucker schaumig rühren.
- 1 Eigelb unterrühren.
- Mehl dazugeben und zu einem glatten Teig verkneten.
- 1 Stunde kalt stellen.
- Teig ausrollen und erstes Blech Kekse ausstechen.
- Kekse auf Backblech legen und 10 Minuten backen.
- Teig ausrollen und zweites Blech Kekse ausstechen.
- Kekse auf Backblech legen und 10 Minuten backen.
- Zutaten für den Zuckerguss mischen
- Kekse mit Zuckerguss bestreichen



Algorithmus strukturieren

Teig herstellen

- Margarine und Zucker schaumig rühren.
- 1 Eigelb unterrühren.
- Mehl dazugeben und zu einem glatten Teig verkneten.
- 1 Stunde kalt stellen.

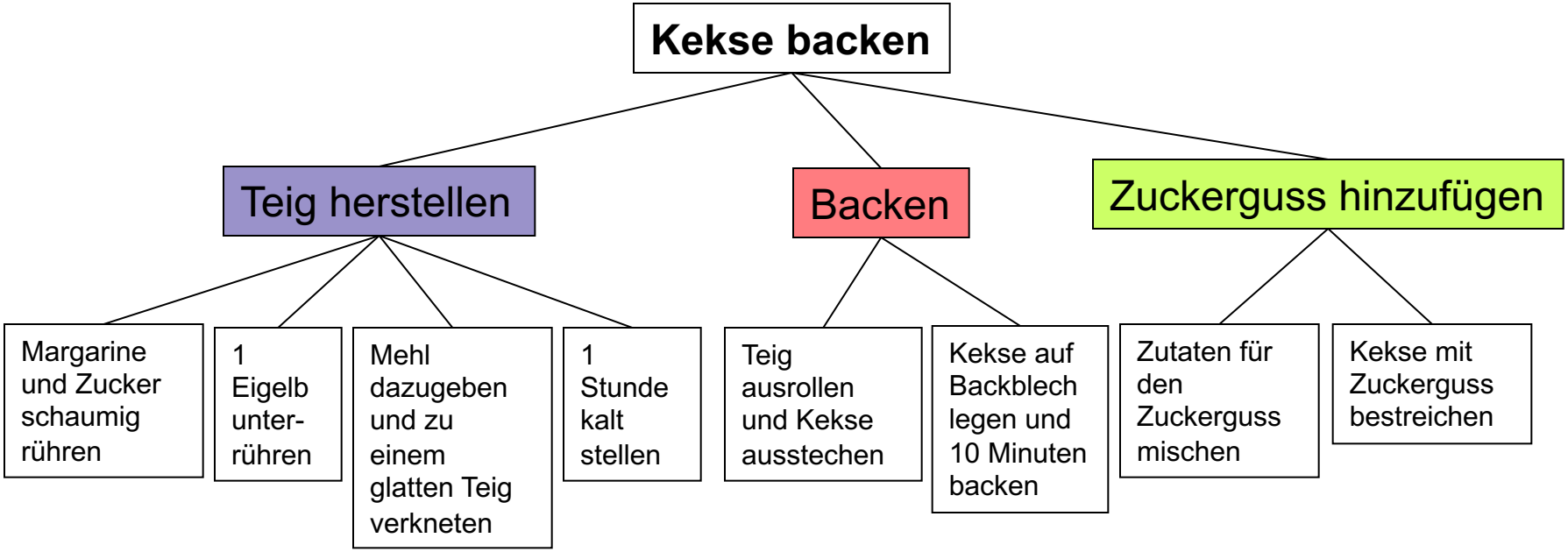
Backen

- Teig ausrollen und Kekse ausstechen.
- Kekse auf Backblech legen und 10 Minuten backen.

Zuckerguss hinzufügen

- Zutaten für den Zuckerguss mischen
- Kekse mit Zuckerguss bestreichen

Strukturierter Algorithmus „Kekse backen“



Dekomposition / Zerlegung

Dekomposition / Zerlegung: Trennung in Teile, wobei jeder Teil einfacher als das Ganze ist.

- Kekse-Backen-Beispiel: Zerlegung einer Aufgabe in Teilaufgaben.
  - „Prozedurale Zerlegung“
- Andere Zerlegungsformen später (insb. Zerlegung in Objekte)

Vorteile der Zerlegung

Unstrukturiert

- Margarine und Zucker schaumig rühren.
- 1 Eigelb unterrühren.
- Mehl dazugeben und zu einem glatten Teig verkneten.
- 1 Stunde kalt stellen.
- Teig ausrollen und erstes Blech Kekse ausstechen.
- Kekse auf Backblech legen und 10 Minuten backen.
- Teig ausrollen und zweites Blech Kekse ausstechen.
- Kekse auf Backblech legen und 10 Minuten backen.
- Zutaten für den Zuckerguss mischen
- Kekse mit Zuckerguss bestreichen

Strukturiert

1. Teig herstellen

2a. Erstes Blech backen

2b. Zweites Blech backen

3. Zuckerguss hinzufügen

Der strukturierte Algorithmus ist:

- hierarchisch und dadurch einfacher zu verstehen.
- Operationen auf höheren Ebenen können genutzt werden, um Redundanz zu vermeiden

Ein Programm mit redundantem Code



https://youtu.be/U7rVJKjFN5c

Ein Programm mit redundantem Code

Einen Songtext von Robin Schulz ausgeben:

```
public class TwoMessages {
    public static void main(String[] args) {
        System.out.println("As the sun goes down");
        System.out.println("Hey");
        System.out.println();
        System.out.println("As the sun goes down");
        System.out.println("Hey");
    }
}
```

Ausgabe:

```
As the sun goes down
Hey

As the sun goes down
Hey
```

Wir geben die gleichen Songzeilen zwei Mal aus.

Ein Programm mit redundantem Code

Redundanz: Die gleiche Folge von Anweisungen taucht mehrfach in einem Programm auf.

- Redundanz vermeiden durch Strukturierung des Programms in mehrere, wiederverwendbare statische Methoden

Statische Methoden

Statische Methode: Eine Folge von Statements mit einem Namen.

Prozedurale Zerlegung: Zerlegung eines Problems in Methoden.

- Eine statische Methode nutzt man wie folgt:
  - 1. Methode deklarieren (Rezept hinschreiben)
    - Schreibe eine Folge von Statements und vergib einen Namen
  - 2. Methode aufrufen (Nach Rezept backen)
    - Dem Programm mitteilen, dass es die Methode ausführen soll.
- Vorteil statischer Methoden:
  - Ein größeres Problem wird strukturiert in kleinen Einheiten beschrieben
  - Redundanz wird durch Wiederverwendung vermieden.

1. Deklaration statischer Methoden

- Syntax der *Deklaration* einer statischen Methode:

```
public class <class name> {

    public static void <method name> () {
        <statement>;
        <statement>;
        ...
        <statement>;
    }
}
```

- Beispiel:

```
public static void printWarning() {
    System.out.println("This product is known to cause");
    System.out.println("cancer in lab rats and humans.");
}
```

2. Aufruf statischer Methoden

- Syntax des *Aufrufs* einer statischen Methode:
  - In einer anderen Methode (z. B. in `main`) schreibe:

    <method name> `();`

- Beispiel:

```
printWarning();
```

- Mehrfacher Aufruf ist möglich:

```
printWarning();
printWarning();
```

Ausgabe:

```
This product is known to cause
cancer in lab rats and humans.
This product is known to cause
cancer in lab rats and humans.
```

Ein Programm mit statischer Methode

```
public class TwoMessages {
    public static void main(String[] args) {
        displayMessage();
        System.out.println();
        displayMessage();
    }

    public static void displayMessage() {
        System.out.println("As the sun goes down");
        System.out.println("Hey");
    }
}
```

Ausgabe:

```
As the sun goes down
Hey

As the sun goes down
Hey
```

Die ursprüngliche Redundanz ist beseitigt.

Methoden rufen andere Methoden auf

- Eine statische Methode ruft eine andere auf:

```
public class MethodsExample {
    public static void main(String[] args) {
        message1();
        message2();
        System.out.println("Done with main.");
    }
    public static void message1() {
        System.out.println("This is message1.");
    }
    public static void message2() {
        System.out.println("This is message2.");
        message1();
        System.out.println("Done with message2.");
    }
}
```

- Ausgabe:

```
This is message1.
This is message2.
This is message1.
Done with message2.
Done with main.
```

Kontrollfluss beim Methodenaufruf

- Wenn eine Methode aufgerufen wird:
  - „springt" die Ausführung in diese Methode,
  - führt dort alle Statements aus, und
  - „springt" zurück zum Statement direkt hinter dem Methodenaufruf.

```
public class MethodsExample {
    public static void main(String[] args) {
        message1();

        message2();

        System.out.println("Dor
    }

    ...
}
```

```
public static void message1() {
    System.out.println("This is message1.");
}
```

```
public static void message2() {
    System.out.println("This is message2.");
    message1();

    System.out.println("Done with message2.");
}
```

Wann setzt man statische Methoden ein?

- Schreiben Sie Statements in eine statische Methode, wenn:
  - die Statements zueinander in enger Beziehung stehen oder
  - die Statements im Programm wiederholt werden.

Sind alle Methoden „public static“?

- Nein.
- Üblich sind eigentlich nicht-static Methoden.
- Außerdem gibt es auch nicht-public Methoden
- Dazu kommen wir aber erst bei der objekt-orientierten Programmierung.
- Merken: public static Methoden sind nur im 1. Semester die Regel.

Vakatseite

Bezeichner

Bezeichner / Identifier: Ein Name für ein Programmelement (Daten, Methode, …).

- Bezeichner erlauben, ein zuvor definiertes Element später im Programm zu verwenden.
- Bezeichner bezeichnen
  - Klassen
  - Methoden
  - Variable und Konstante (später)

Bezeichner in Java

- Java-Bezeichner:
  - Erstes Zeichen: Buchstabe oder Unterstrich (_) oder $
  - Weitere Zeichen: wie erstes Zeichen oder Ziffern
  - Groß-/Kleinschreibung ist relevant
    (`name` ist ein anderer Bezeichner als `Name`)

- Beispiele:
  - gültig:

```
susan          second_place     _myName
TheCure        ANSWER_IS_42     $variable
```

  - ungültig:

```
me+u           49er             question?
side-swipe     hi there         ph.d
jim's          2%milk           suzy@yahoo.com
```

Bezeichner in Java

- Als Buchstabe gelten nicht nur unsere lateinischen Buchstaben aus dem Bereich A-Z und a-z.
- Auch Umlaute und viele weitere Zeichen sind erlaubt (€, ¥), sogar griechische Buchstaben.

- **Wichtige Konvention**: Verwenden Sie nur Buchstaben aus dem Bereich A-Z, a-z und ggf. einen Unterstrich. U. a. also: keine Umlaute.

- **Weitere wichtige Konventionen**:
  - Klassen: "CamelCase" (Beispiel: KundenKonto)
  - Methoden: Großbuchstabe ab dem zweiten Wort (Beispiel: geldAbheben)
  - Variable: wie Methoden (Beispiel: kontoStand)
  - Konstante: Großbuchstaben. Wörter durch _ trennen (Beispiel: DISPO_LIMIT)

Schlüsselwörter

Schlüsselwort / keyword: Ein Bezeichner, der in Java bereits für eine bestimmte Bedeutung reserviert ist, und daher nicht selbst vergeben werden kann.

- Diese Schlüsselwörter kennen wir bereits:
  `public static void class`

- Sie können `public` oder `void` nicht für eine eigene Klasse oder Methode verwenden.
- (Sie könnten `PUBLIC` oder `VoiD` verwenden. Tun Sie aber besser nicht, da es andere Leser Ihres Programms verwirrt.)



- Weitere Schlüsselwörter entnehmen Sie den Leseaufgaben.

Vakatseite

Kommentare

Kommentar: Eine vom Programmierer eingebrachte Notiz im Quellcode, die den Code leichter verständlich macht.

- Kommentare werden bei der Programmausführung nicht ausgeführt.
- Viele Editoren (u. a. jEdit) zeigen Kommentare in einer anderen Farbe an.

Kommentare

- Syntax für Kommentare:

  /* ***<comment text; may span multiple lines>*** */

  oder,

  // ***<comment text, on one line>***

- Beispiele:

```
/* A comment goes here. */

/* It can even span
   multiple lines. */

// This is a one-line comment.
```

Wann sollten Sie Kommentare verwenden?

- Am Anfang jeder Datei
  - Autor
  - Beschreibung, was das Programm kann
- Am Anfang jeder Methode
  - Beschreibung des Verhaltens
- Innerhalb von Methoden
  - Um komplexe Codestücke zu erläutern
  - Besser: keine komplexen Codestücke schreiben (selbsterklärender Code)

Konvention: Kommentarformat `/** … */`, d.h. zwei einleitende `*`

Häufig mit dem Zeilenkommentar `//`

- Kommentare sind wichtig!
  - Programme mit tausenden von Programmzeilen und hunderten von Methoden werden ohne Kommentare unübersichtlich.
  - Wenn mehrere Programmierer zusammen arbeiten, helfen Kommentare beim Verstehen des Codes der KollegInnen.

Beispiel

```
/** Susie Student
    PR1, WS2015/16
    Dieses Programm druckt den Songtext meines Lieblingssongs aus! */
public class MeinLieblingssong {
    /** Startet das Programm und sorgt für den Ausdruck der Ausgabe auf
        die Console. */
    public static void main(String[] args) {
        singen();

        // Trenne die beiden Strophen durch eine Leerzeile
        System.out.println();

        singen();
    }

    /** Zeigt die erste und zweite Strophe des Songs an. */
    public static void singen() {
        System.out.println("Now this is the story all about how");
        System.out.println("My life got flipped turned upside-down");
    }
}
```

Wie kommentiert man Methoden?

- Keine detaillierte Beschreibung der Syntax / der Anweisungen.
- Stattdessen: kurze Beschreibung des beobachtbaren Verhaltens, wenn die Methode ausgeführt wird.

  – Beispiel:

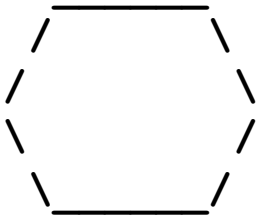
```
/** Diese Methode druckt die erste Strophe meines Lieblingssongs.
    Leerzeilen trennen die einzelnen Strophenteile. */
public static void strophe1() {
    System.out.println("Now this is the story all about how");
    System.out.println("My life got flipped turned upside-down");
    System.out.println();
    System.out.println("And I'd like to take a minute,");
    System.out.println("just sit right there");
    System.out.println("I'll tell you how I became the prince");
    System.out.println("of a town called Bel-Air");
}
```

Vakatseite

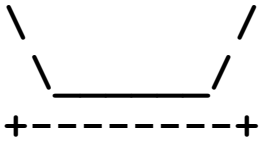
Eine Aufgabe “aus dem richtigen Leben”

- Wir schreiben ein Programm, das folgende Dinge des täglichen Lebens ausgibt:

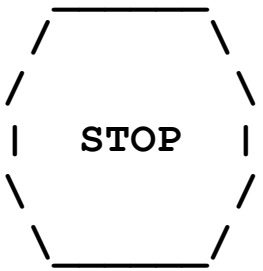














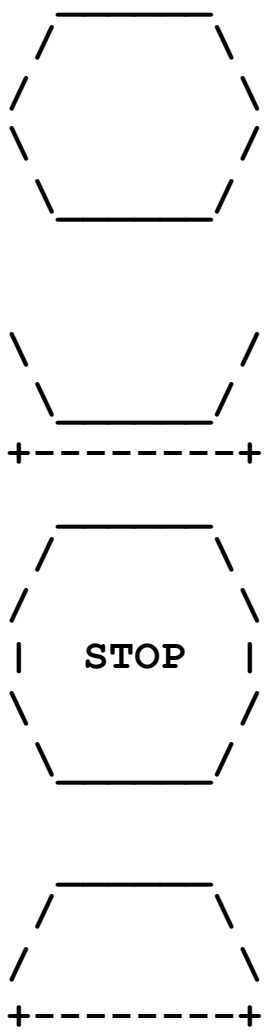


Wie geht man vor?

- Wir müssen ein Problem lösen.
- Problemlösung in mehreren Schritten.
- Zunächst eine einfache, unstrukturierte Lösung, die zu einem ersten Erfolg führt.
- Diese Lösung nach und nach verbessern (strukturieren).

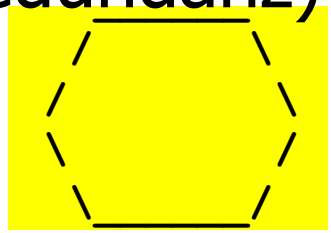
Erste Programmversion (Unstrukturiert)

- Eine main-Methode
- Erwartete Ausgabe in die main-Methode kopieren.
- Jede Zeile mit der System.out.println-Syntax umhüllen.
- Ausführen und Ausgabe prüfen.



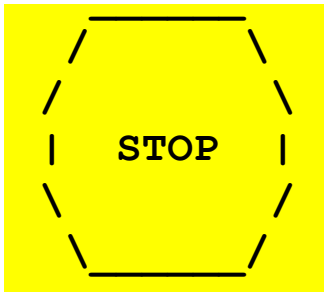
Zweite Programmversion (Strukturiert, aber immer noch mit Redundanz)

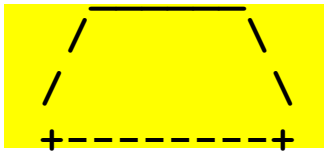
- Identifiziere die Struktur der Ausgabe.



- Teile die main-Methode in einige statische Methoden (je eine Methode für jeden Ausgabeteil).





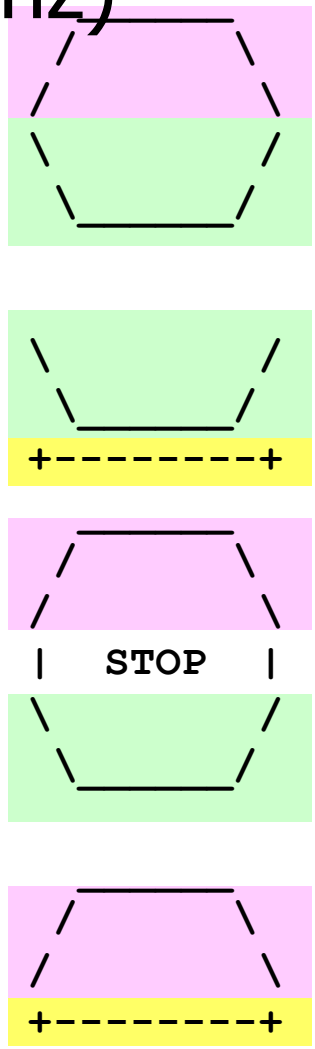


Dritte Programmversion (Strukturiert, ohne Redundanz)

- Identifiziere jedwede Redundanz in der Ausgabe

- Teile das Programm in weitere statische Methoden ein, um Redundanz so weit wie möglich zu eliminieren.

- Kommentiere das Programm zur Verbesserung der Lesbarkeit.



Vakatseite